

✈ Ortho4XP V3.0

Documentation complète — Juin 2026

Modernisation • Colorimétrie calibrée • Fusion sources • Corrections DSF

Roland (Ypsos) — macOS / Windows / Linux

Concept & design : Roland pseudo Ypsos | Codage : Claude (Anthropic AI)

Version v18 — Juin 2026



MÉMORANDUM — Origine et transparence

Ortho4XP V3.0 est une évolution personnelle et moderne de la version 1.40.

- Logiciel original : créé par Oscar Pilote → github.com/oscarpilote/Ortho4XP
- Version 1.40 maintenue : fork par Shred86 → github.com/shred86/Ortho4XP
- Cette V2 : refonte complète développée par Roland (Ypsos) avec l'aide de Claude (Anthropic AI).

En mars 2026, j'ai contacté Oscar Pilote et la communauté (Issue GitHub #299, Topic X-Plane.org). Réponse : « Tu fais ce que tu veux, tu es libre ». Aucun accès à une branche n'ayant été accordé, cet espace dédié a été créé afin que la version V2 soit claire, indépendante et accessible à tous.

— PARTIE 1 —

Table des matières

PARTIE 1 — POUR L'UTILISATEUR

1. Ce qu'apporte Ortho4XP V3.0 vs V1.40 ?
2. Nouveautés et modernisation V3
3. Installation — flux complet
4. Prérequis système par plateforme
5. Color Normalize & Saturation
6. Textures mer côtière — O4_Sea_Texture
7. Masque côtier naturel — O4_Coastal_Manager
8. Outils de diagnostic — O4_Benchmark
9. Color Check — fonctionnement et Fusion Dégradé
10. Workflow recommandé et validé

PARTIE 2 — TECHNIQUE

11. Processus interne JPG → PNG → DDS
12. Fichiers spéciaux — Pour le développeur
13. Fichiers Ortho4XP modifiés et ajoutés
 - 13.1 Nouveaux fichiers V3
 - 13.2 Correctifs Python 3.12
 - 13.3 Fonctionnalités V3
14. Modules Python et versions venv/
15. Architecture des fichiers ORTHO4XP_V3/
16. Historique des versions et modifications

PARTIE 3 — LICENCE & CRÉDITS

17. Avertissement légal — ORTHO4XP_V3
18. Remerciements

POUR L'UTILISATEUR

Découverte, installation et utilisation des nouvelles fonctions

Ce qu'apporte Ortho4XP V3.0 vs V1.40 ?

 PERFORMANCE	 STABILITÉ AU LANCEMENT
• Python 3.12 — 15 à 20 % plus rapide sur les calculs de mesh • Modules numpy, Pillow, scipy compilés nativement arm64 • Environnement venv isolé — aucun conflit de modules • 10 slots de conversion simultanés (selon CPU)	• Sentinelle CPU/RAM au lancement — détecte le matériel • Calcul automatique des slots sécurisés (CPU - 2 cœurs réservés) • RAM limitée à 75 % max — évite les crashes mémoire • Config écrite automatiquement dans Ortho4XP.cfg
 COLORIMÉTRIE calibrée 48 753 JPG	 ALTIMÉTRIE & INSTALLATION
• Color Normalize : correction sRGB EN MÉMOIRE sur JPEG avant création et azssemblage duy OPNG et création → DDS • Référent : cube sRGB_COLORIMÉTRIE calibrée sur 48 753 JPG). — cible théorique universelle • Color Check : détection et correction des dominantes R/G/B • Curseurs R/G/B + Luminance + Contraste + Netteté + O4_Color_Apply • Fondu grains de sable exponentiel	• rasterio dans venv — 100% autonome, aucune dépendance système • Installation double-clic — aucun terminal pour l'utilisateur • Lanceur natif .app/.vbs/.desktop créé automatiquement • Dossier entièrement autonome — déplaçable sur disque externe

1. Comparatif V1.40 vs V3.0

Domaine	Ortho4XP 1.40	Ortho4XP V3.0
Python	3.10 / 3.11	3.12 — 15 à 20% plus rapide sur les mesh
Installation	Terminal obligatoire	Double-clic — aucun terminal requis
GDAL / Altimétrie	osgeo.gdal via Homebrew système (conflit venv)	rasterio dans venv — 100% autonome, aucun conflit
Modules Python	Installés dans le système global	Isolés dans venv/ — aucun conflit, déplaçable
Lanceur	Aucun ou script .command	Binaire C universel arm64+x86_64
Interface	Standard	4K adaptée, Color Normalize 2 lignes, boutons uniformes
Colorimétrie	Basique	Calibrée 48 753 JPG Europe, fondu grains de sable
Preview OSM	tile.openstreetmap.org (403 bloqué)	tile.openstreetmap.de (actif + déplacement )
Python 3.12	Plusieurs erreurs runtime (imports, API obsolètes)	7 fichiers corrigés — 0 erreur
Désinstallation	Laisse des traces système	Supprimer le dossier = désinstallation complète

2. Nouveautés et modernisation V3

Ortho4XP V3.0 est une refonte complète du système d'installation et de lancement. L'objectif principal : permettre à tout utilisateur d'installer et lancer Ortho4XP sans jamais ouvrir un terminal.

- ✓ Python 3.12 — 15 à 20% plus rapide sur les calculs de mesh
- ✓ Sentinelle CPU/RAM au lancement — détecte le matériel, calcule automatiquement les slots sécurisés
- ✓ Environnement isolé venv/ — rien n'est modifié sur le système hôte
- ✓ Launcher graphique avec détection automatique et installation en un clic
- ✓ Compatible macOS (Apple Silicon M1–M4 et Intel), Windows 10/11, Linux
- ✓ Dossier Ortho4XP entièrement autonome — déplaçable sur disque externe
- ✓ Lanceur natif .app/.vbs/.desktop créé automatiquement par create_launcher.py
- ✓ Interface adaptée 4K — polices agrandies automatiquement, fenêtre protégée
- ✓ rasterio dans venv — remplace osgeo.gdal, 100% autonome, aucune dépendance système
- ✓ Tuiles produites avec succès — validation complète X-Plane 12

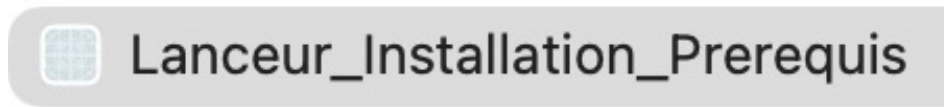
3. Installation — flux complet

7.1 Fichiers du système d'installation

Fichier	Rôle	Exécuté par
create_launcher_prerequis.py	Génère Lanceur_Installation_Prerequis.app / .vbs / .desktop selon la plateforme	Développeur — 1 seule fois avant distribution
Lanceur_Installation_Prerequis.app	Lance INSTALL_PREREQUIS.py — binaire C universel, pas besoin de Python	Utilisateur — premier double-clic
INSTALL_PREREQUIS.py	Fenêtre verte — détecte le système, installe Python/Homebrew/venv/modules	Lanceur_Installation_Prerequis
create_launcher_ORTHO.py	Génère Lanceur ORTHO4XP.app / .vbs / .desktop via le bouton du Launcher	Utilisateur via bouton Launcher
Lanceur ORTHO4XP.app	Lance directement Ortho4XP_Launcher.py via venv — usage quotidien	Utilisateur — double-clic quotidien
Ortho4XP_Launcher.py	Interface principale — installation dépendances et lancement Ortho	Lanceur ORTHO4XP

7.2 Flux première installation

1. Double-clic sur Lanceur_Installation_Prerequis.app (binaire C — pas besoin de Python)



2. La fenêtre détecte macOS/Linux/Windows, installe Homebrew + Python 3.12 si absent, puis installe tous les modules dans venv/ (dont rasterio pour l'altimétrie)
3. Cliquer sur Installer les Modules



Fenêtre INSTALL_PREREQUIS — détection automatique macOS/arm64 et installation

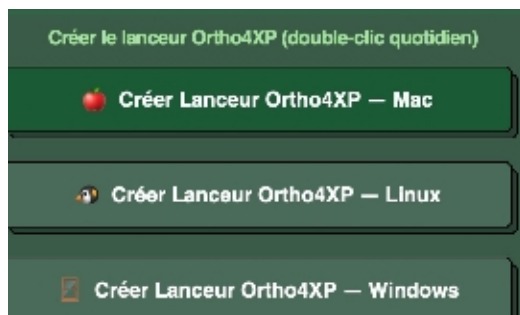
4. Cliquer sur le bouton correspondant à votre système → Installe tous les modules dans venv/ (dont rasterio pour l'altimétrie)



✓ rasterio inclut ses propres librairies GDAL compilées. Aucune installation GDAL système requise sur aucune plateforme (Mac, Linux, Windows).

⚠ GDAL via osgeo/Homebrew définitivement retiré — incompatibilité symbol _CPLDefaultErrorHandler entre Homebrew 3.8.4 et pip gdal 3.12.3 dans le venv.

5. Dans la même fenêtre cliquer sur le bouton de votre système pour créer le futur lanceur d'Ortho4XP_V2 : bouton 'Créer Lanceur Mac/Linux/Win'



Exemple Lanceur Mac : Usage quotidien : double-clic sur Lanceur ORTHO4XP.app
(peut être placé sur le bureau, dans le Dok etc...)



6. Lance Ortho4XP_Launcher.py (*au premier lancement il fait être très patient*)



4. Prérequis système par plateforme

Double-cliquer sur Lanceur_Installation_Prerequis — la fenêtre détecte automatiquement le système, vérifie les versions Python et modules nécessaires, et installe ce qui est absent ou obsolète.

5.1 macOS (Apple Silicon M1/M2/M3/M4 + Intel)

Composant	Version / Commande / Notes
macOS	12.0 minimum (macOS 14 Sonoma recommandé)
Xcode Command Line Tools	xcode-select --install — requis pour compiler le binaire C
Homebrew	https://brew.sh — installé automatiquement par INSTALL_PREREQUIS si absent
Python 3.12	brew install python@3.12
python-tk@3.12	brew install python-tk@3.12 (interface graphique tkinter)
spatialindex, p7zip, proj, libspatialite	brew install ... — installés via le bouton macOS du Launcher

⚠ *Apple Silicon (M1-M4) : Homebrew s'installe dans /opt/homebrew. Sur Intel : /usr/local. Les deux sont supportés automatiquement.*

5.2 Linux — Debian/Ubuntu

Composant	Notes
python3-pip, python3-venv, python3-tk	Installés automatiquement via apt
p7zip-full	Compression 7z requise pour les archives DSF
libspatialindex-dev, proj-bin, libgeos-dev	Bibliothèques géospatiales système
python3-numpy, python3-pyproj, python3-shapely, python3-rtree, python3-pil	Paquets système Python

5.3 Linux — Arch/Manjaro

Composant	Notes
python, python-pip, tk	Installés via pacman
spatialindex, p7zip, proj	Installés via pacman

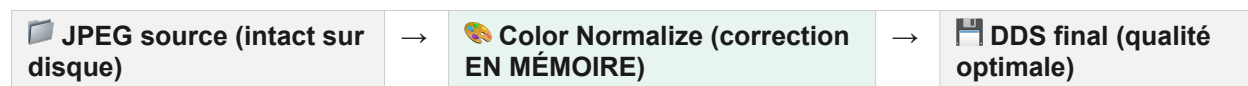
5.4 Windows 10/11 (64-bit)

Composant	Notes
Python 3.12	https://www.python.org — cocher 'Add Python to PATH' et 'Install pip', ou via winget
7-Zip	https://www.7-zip.org — ou installé automatiquement
Tous les modules	Installés dans venv/ via pip automatiquement

5. Color Normalize & Saturation — fonctionnement et interface

11.1 Principe

Pendant chaque Build, juste avant de convertir le JPEG en DDS, Ortho4XP appelle Color Normalize. Celui-ci analyse le JPEG source et le corrige statistiquement en mémoire pour l'aligner vers le cube de référence calibré sur 48 753 JPG Europe. Le JPEG source reste intact sur le disque.



✓ Le JPEG source n'est jamais modifié sur le disque. Le DDS existant est supprimé puis recréé. La correction s'applique uniquement en mémoire pendant la conversion. Meilleur résultat : on travaille sur la donnée source haute qualité avant compression DDS.

11.2 Cube de référence calibré sur 48 753 JPG

- Analyse réelle de 48 753 JPG Europe (France, Belgique, Hollande, Allemagne, Suisse, Luxembourg, Vendée...)
- Moyenne globale : R=86.5 G=96.5 B=86.9
- Gamma réel : $\gamma_r=0.63$ $\gamma_g=0.71$ $\gamma_b=0.63$ — orthophotos aériennes naturellement comprimées
- R/B = 0.995 → quasi neutre D65, pas de dominante chromatique
- CORRECTION_STRENGTH = 0.30 — correction douce, harmonise sans écraser

11.3 Pipeline de correction complet

7. Color Normalize : chaque JPG source normalisé INDIVIDUELLEMENT avant assemblage
8. Assemblage grains de sable : fusion des sources avec correction croisée vers cible commune
9. Color Apply (.ccorr) : corrections résiduelles appliquées APRÈS normalisation
10. Conversion DDS : PNG → DDS DXT1 sans flag sRGB

11.4 Confirmation pipeline X-Plane — analyse DDS

Analyse des headers DDS des textures mesh X-Plane 12 — résultats révélateurs :

Constat	Explication et conséquence
Pas de flag sRGB — tous en DXT5	X-Plane traite les DDS mesh et les DDS orthophotos de la même façon : pixels affichés tels quels, sans conversion gamma. Nos orthophotos sont dans le bon pipeline.
Gamma réel très bas : 0.38 à 0.90	Intentionnel — Laminar Research encode ces textures sombres pour compenser l'éclairage dynamique X-Plane (HDR, cycles jour/nuit). X-Plane éclaircit le rendu en temps réel via son moteur.
Nos orthophotos gamma ~0.63	Même plage que les textures mesh (0.38–0.90). Cela explique pourquoi le rendu est correct sans correction gamma supplémentaire dans le pipeline V2.
R/B > 1 sur toutes les textures mesh (1.07 à 1.74)	Dominante chaude intentionnelle dans X-Plane. Nos orthophotos à R/B=0.995 (neutre D65) apparaissent légèrement plus froides/vertes — visible à la jointure orthophoto/zones sans couverture.
Conclusion	Nos DDS orthophotos sont correctement calibrés pour le pipeline X-Plane. La différence de teinte aux jointures mesh/ortho est normale et inhérente aux sources de données différentes — elle ne peut pas être corrigée par Color Normalize.

11.5 La barre Color Normalize dans l'interface

Ligne 1 — Color Normalize | Enable | Strength: ←slider→ 30% | Réf: Calibré_48753_JPG_Europe
Ligne 2 — Saturation : Enable | Boost: ←slider→ 0-200% | 0%=gris 100%=réf.48753JPG 200%=x2
Ligne 3 — RGB adjustments, sharpness, saturation (Color Check) | ⌚ Timeline | RAM: live

Élément	Description
Checkbox Enable ON/OFF	ON : corrections actives. Se grise quand Color Check est ouvert — se réactive à la fermeture.
Curseur Strength 0–100%	Intensité globale de toutes les corrections. 30% recommandé (calibré 48 753 JPG).
Réf: label	Bleu = référent global calibré actif. Orange = référent temporaire (DDS chargé pour la tuile courante).
Saturation Enable / Boost	Checkbox Enable ON/OFF : active le curseur Boost. Curseur Boost 0–200% : 0% = image totalement désaturée (niveaux de gris), 100% = saturation de référence calibrée sur 48 753 JPG (valeur par défaut), 200% = saturation doublée. Appliqué en mémoire via PIL ImageEnhance.Color — le JPG source reste intact sur le disque.
Color Check...	Ouvre la fenêtre de détection des jointures. Toujours actif même si Enable est OFF.

11.6 Fournisseurs personnels — Imagery

V3.2 ajoute la possibilité de créer ses propres fournisseurs d'images satellite directement depuis l'interface, sans modifier aucun fichier source. Les providers personnels apparaissent immédiatement dans la liste Imagery et sont utilisables comme n'importe quel fournisseur standard.

Principe de fonctionnement

Chaque provider personnel est un fichier .lay de type TMS (webmercator) sauvegardé dans le dossier Providers/Personnel/. L'entrée spéciale « Personnel » en fin de liste Imagery ouvre la fenêtre de gestion — elle n'est jamais écrite dans le cfg comme provider réel.

Élément de l'interface	Description
Entrée « Personnel » dans Imagery	Toujours présente en fin de liste Imagery. Un clic ouvre la fenêtre de gestion. N'est jamais écrite dans Ortho4XP.cfg comme provider actif.
Nom du provider	Alphanumérique, tirets et underscores autorisés. Les noms réservés (OSM, SEA, PATCH, ZonePhoto, Personnel, Custom) sont refusés. Utilisé comme nom de dossier dans Orthophotos/ et comme code provider.
URL TMS	URL du service TMS (type webmercator), généralement au format <code>https://.../{z}/{x}/{y}.jpg</code> ou similaire. Sauvegardée dans le fichier .lay généré (url_template).
Bouton Sauvegarder	Crée le fichier .lay dans Providers/Personnel/. Le provider apparaît immédiatement dans la liste Imagery et est sélectionné automatiquement.
Modifier / Supprimer	Sélectionner un provider dans la liste, modifier l'URL puis cliquer Modifier. Supprimer efface le .lay du disque et retire le provider de la liste Imagery immédiatement.
Fichier généré	Providers/Personnel/NOM.lay — contient <code>request_type=tms</code> , <code>grid_type=webmercator</code> , <code>url_template</code> , <code>in_GUI=True</code> . Compatible avec tous les ZL et toutes les tuiles.

6. Textures mer côtière — O4_Sea_Texture

Le problème corrigé

Dans X-Plane 12, les tuiles qui couvrent une côte (terre + mer) pouvaient afficher deux problèmes visuels majeurs sur la zone de mer :

- Un damier bleu — la texture mer d'origine X-Plane apparaissait en damier sur la zone côtière
- Des triangles transparents — certaines zones de mer côtière s'affichaient en transparent, laissant voir le sol 3D sous-jacent

Ces problèmes venaient du fait qu'Ortho4XP ne générait pas de texture satellite pour les zones de mer dans les JPG côtiers — ces zones étaient laissées en noir (nodata). X-Plane affichait alors sa propre texture bleue par défaut, avec un résultat visuel incohérent.

Ce que fait O4_Sea_Texture

O4_Sea_Texture analyse chaque JPG côtier et détecte les zones nodata (pixels noirs ou blancs uniformes). Quand une zone nodata est trouvée, il reconstruit une texture réaliste en récupérant la vraie couleur de l'eau depuis les JPG voisins valides — la couleur satellite réelle de la mer à cet endroit (verte, turquoise ou bleue selon la zone).

Le JPG corrigé est sauvegardé comme JPG-Patch dans Orthophotos/JPG-Patch/{tuile}/PATCH_{z}/. Ortho4XP l'utilise automatiquement comme source supplémentaire lors du Build Step 3.

✓ Cette correction est entièrement automatique — elle s'active au Step 2.1 (build_sea_patches). Vous n'avez rien à configurer.

⚠ Les JPG sources ne sont jamais modifiés. Les JPG-Patch sont des fichiers séparés dans un dossier dédié.

Avant / Après

Avant V3.2 — mer côtière : damier bleu ou triangles transparents dans X-Plane 12.

Après V3.2 — mer côtière : vraie couleur satellite continue, sans jointure visible avec la terre.

Pipeline automatique

- Step 1 — build_poly_file : génération du fichier de découpe de la tuile
- Step 2 — build_mesh : génération du mesh 3D de la tuile
- Step 2.1 — build_sea_patches : O4_Sea_Texture scanne les JPG côtiers, détecte les zones nodata, génère les JPG-Patch avec la vraie couleur mer
- Step 2.5 — build_masks : génération des masques eau, puis appel O4_Coastal_Manager pour affiner le bord côtier
- Step 3 — build_tile : assemblage DDS — le provider PATCH est automatiquement injecté, les JPG-Patch sont utilisés comme source de fond

✓ Si vous supprimez le dossier JPG-Patch avant un rebuild, O4_Sea_Texture les régénère automatiquement.

7. Masque côtier naturel — O4_Coastal_Manager

Le problème corrigé

Dans la version originale Ortho4XP 1.40, la limite entre la terre et la mer dans X-Plane était un bord mathématiquement net — une ligne droite ou légèrement arrondie. En vol, ce bord artificiel était clairement visible, surtout sur les côtes avec du sable ou des rochers.

L'idée vient de Roland et de la méthode Jojo : les masques manuels produits par Jojo avaient un bord vague/écume naturel qui rendait la transition terre/mer beaucoup plus réaliste. O4_Coastal_Manager automatise cette approche.

Ce que fait O4_Coastal_Manager

Après qu'Ortho4XP génère le masque PNG côtier, O4_Coastal_Manager applique un post-traitement sur ce masque :

- Détection du bord côtier dans le PNG (frontière entre la zone blanche = terre et la zone noire = mer)
- Irrégularisation du bord par bruit fractal multi-octave — les ondulations imitent les vagues et l'écume
- Bande d'écume semi-transparente côté mer — transition douce et réaliste
- Protection absolue : l'intérieur de la terre (blanc plein) et l'eau profonde (noir plein) ne sont jamais touchés — seul le bord est traité

✓ Le post-traitement est entièrement automatique et invisible — vous n'avez rien à faire.

⚠ O4_Coastal_Manager ne génère pas de masque — il affine uniquement les masques déjà créés par Ortho4XP. Si aucun masque côtier n'existe, il ne fait rien.

Paramètres (validés)

ECUME_WIDTH_PX = 30 — largeur de la bande d'écume en pixels sur le masque ZL15 (environ 120 mètres réels). Ce paramètre est validé et ne nécessite pas d'ajustement pour la majorité des côtes.

NOISE_AMPLITUDE_PX = 20 — amplitude maximale des irrégularités du bord côtier en pixels (environ 80 mètres réels). Produit un bord naturellement ondulé sans être exagéré.

Règle ZonePhoto

(Uniquement pour les membre du groupe de développeur de tuiles de xPlanePiloiteControle)

Si la tuile utilise ZonePhoto avec une zone d'étendue définie (`extent_code` ≠ `global`), O4_Coastal_Manager ne génère pas de masque côtier — ZonePhoto fournit déjà ses propres JPG mixtes terre+mer et la génération automatique de masque produirait un résultat incorrect.

8. Outils de diagnostic — O4_Benchmark

À quoi ça sert

O4_Benchmark est un module autonome de diagnostic et de mesure de performance pour Ortho4XP V3. Il ne modifie aucune tuile et n'affecte pas le Build. Il sert à comprendre ce qui se passe pendant la génération et à identifier les goulots d'étranglement.

Benchmark CPU

Mesure les performances des opérations lourdes sur votre machine — traitement d'images, calculs de mesh, opérations colorimétriques. Permet de savoir si votre CPU est le facteur limitant, et de comparer différentes configurations.

- Lance une série de tests sur les opérations réelles d'Ortho4XP
- Affiche les temps de traitement par opération
- Identifie les opérations les plus coûteuses sur votre machine

Timeline

Chronomètre les étapes du Build en temps réel. Chaque step est mesuré et rapporté avec son temps d'exécution.

- `start("Step X")` — marque le début d'une étape
- `end("Step X")` — marque la fin de l'étape
- `report()` — affiche un résumé chronologique de toutes les étapes

✔ *La Timeline est particulièrement utile pour identifier pourquoi un Build particulier est lent.*

DebugViz — Visualisations de diagnostic

DebugViz génère des images de diagnostic pour visualiser ce qu'Ortho4XP calcule en interne :

- Seam risk map — carte du risque de jointures visibles entre sources
- Blur map — carte des zones de flou appliquées par les algorithmes de fusion
- Color transfer map — visualisation des corrections colorimétriques appliquées

⚠ *O4_Benchmark est un module autonome — il ne s'active que si vous l'importez explicitement. Il n'a aucun impact sur les Builds normaux.*

9. Color Check — fonctionnement, interface et Fusion Dégradé

12.1 Principe général

Color Check détecte les DDS à dominante colorimétrique et mémorise les corrections dans un fichier `.ccorr`. Au Build suivant, Ortho relit les JPEG sources et génère de nouveaux DDS propres avec les corrections mémorisées. Le JPEG source n'est jamais modifié.

Comment fonctionne Color Check

Color Normalize applique une correction identique à toutes les photos — c'est une harmonisation globale vers le cube de référence calibré. Color Check va plus loin : il analyse chaque DDS individuellement et détecte ceux qui ont encore une dominante colorimétrique résiduelle (trop rouge, trop vert, trop bleu). Vous corrigez uniquement les DDS qui en ont besoin, groupe par groupe, avec un aperçu avant/après en temps réel.

C'est du sur-mesure par groupe — pas une correction générique appliquée d'un coup aux 800 DDS de la tuile.

Workflow pas à pas

- Cliquer Color Check... → la fenêtre s'ouvre, Color Normalize se désactive automatiquement
- Section ① — Cliquer Identifier dominantes : scan de tous les DDS en parallèle
- La liste gauche affiche les DDS à dominante, organisés par couche ZL et extend — chaque entrée indique le numéro JPG, la couche ZL, la couleur dominante et sa valeur
- Utiliser le champ de recherche pour retrouver un DDS par numéro JPG ou nom
- Cliquer sur un DDS dans la liste → il s'affiche dans la fenêtre Image source
- Ajuster le curseur R/G/B correspondant (ex. G -11 pour corriger une dominante verte de 11 points) → aperçu Après en temps réel
- Si le résultat convient → Appliquer au groupe → tous les DDS du même groupe reçoivent la même correction
- Cliquer Lancer Build → les DDS du groupe sont supprimés et recréés avec la correction
- Cliquer Rescanner → vérifier que la dominante a disparu → recommencer pour le groupe suivant
- Fermer Color Check → Color Normalize se réactive

Section ② — Fusion Dégradé et jointures entre sources

Pour les tuiles avec plusieurs fournisseurs (frontière France/Belgique, etc.), Color Check permet aussi de régler la fusion entre sources. Sélectionner un DDS à jointure visible → cliquer Fusion Preview → choisir le rayon optimal par double-clic.

- Rayon 48 px — différences légères entre sources
- Rayon 64 px — recommandé pour la plupart des frontières
- Rayon 96 px — différences marquées, grand écart colorimétrique
- Rayon 128 px — maximum, cas extrêmes

La FusionPreviewWindow affiche aussi le ΔE (écart colorimétrique mesuré entre les deux sources) avec des conseils adaptatifs : faible / modéré / fort / critique. Le rayon est automatiquement majoré au Build si l'écart est fort ($\times 1.3$ à $\times 2.0$ selon ΔE , plafonné ZL18+).

Bouton Générer .comb seam

Si une jointure persiste malgré un rayon élevé, le bouton Générer .comb seam détecte automatiquement la ligne de jointure dans le DDS sélectionné et génère un fichier .comb de protection sur la zone. Ce masque est pris en compte au Build suivant pour protéger la zone de correction.

Mode batch preview

Le mode batch permet d'évaluer l'impact d'une correction sur une couche ZL entière avant de l'appliquer — utile pour vérifier qu'une correction de groupe n'introduit pas de nouvelle dominante sur les DDS adjacents.

12.2 Section ① — Correction des dominantes

Scan de tous les DDS de la tuile en 8 threads parallèles. Détection des dominantes (seuil > 8 pts d'écart). Correction additive douce vers le cube de référence.

Élément	Description
Liste des DDS	Uniquement les DDS à dominante. Format : [R/G/B +Xpt] nom.dds
Exporter liste	Sauvegarde toutes les dominantes dans un fichier texte
Aperçu Avant / Après	Vignettes comparatives. L'Après se met à jour en temps réel
Curseurs R/G/B	Correction additive par canal, de -60 à +60 points
Curseurs Lum/Cont R/G/B	Luminance et contraste par canal, de -50 à +50 points
Curseur Netteté	Netteté 0 à 300 via PIL ImageEnhance.Sharpness. Inclus dans Reset
Auto-détecter	Calcule la correction exacte pour le DDS sélectionné
Supprimer TOUS	Traite toute la liste avec confirmation
Lancer Build	S'active après suppression. Lance le Build sans quitter la fenêtre
Rescanner	Relance l'analyse après un Build pour vérifier
Barre de statut	X DDS scannés — Y dominantes — Z propres — W erreurs (seuil ≥ N pts)

12.3 Section ② — Fusion Dégradé (algorithme grains de sable)

Pourquoi le fondu cosinus ne suffit pas

La première approche pour la V2, utilisait une bande cosinus entre les deux sources. Le problème : une bande cosinus a DEUX bords — un bord intérieur et un bord extérieur. Même si l'intérieur est ondulé par le bruit de Perlin, le bord extérieur reste une limite nette que l'œil humain capte immédiatement en vol.

✗ Bande cosinus (ancien)	✓ Grains de sable (V2)
Deux bords nets (intérieur + extérieur). L'œil voit la bande même avec Perlin. Limite toujours détectable en vol.	Pas de bord. La probabilité décroît exponentiellement — la transition s'évanouit naturellement, jamais brutalement.

Principe de l'algorithme grains de sable

L'analogie de la nature : une forêt qui se transforme en champ n'a pas une ligne franche — elle a des arbres de plus en plus espacés, sur des centaines de mètres. L'œil ne perçoit jamais de début ou de fin. Ortho4XP V2 applique la même logique aux jointures entre sources orthophotos :

x

Élément de l'algorithme	Fonctionnement détaillé
Principe fondamental	Chaque pixel est 100% source A OU 100% source B — zéro flou, zéro interpolation de couleur. Seule la PROBABILITÉ d'appartenir à chaque source varie.
Calcul de la distance signée	scipy.ndimage.distance_transform_edt calculé à 1/8 de résolution (512×512 au lieu de 4096×4096) pour la vitesse. Distance positive = côté source A, négative = côté source B.
Probabilité exponentielle	prob_A = 1 / (1 + exp(-k × distance)). Constante k = log(3) / rayon. À distance=0 : prob=50%. À distance=+rayon : prob=85%. À distance=+3×rayon : prob=97%. Jamais 100% — les grains s'évaporent progressivement.
Bruit Perlin global	Grille 256×256 cellules de bruit, seed = hash(provider_code + zoomlevel). Partagée par TOUS les DDS du build → ondulations continues à travers les tuiles adjacentes — aucune discontinuité aux bords des DDS.
Tirage pixel par pixel	np.random.uniform(0,1) comparé à prob_A pour chaque pixel. Si threshold < prob_A → pixel source A, sinon source B. Seed locale reproductible → même résultat à chaque rebuild.
Correction croisée (v17)	Avant le tirage, les deux sources sont poussées vers une cible commune (moyenne 50/50). Force = 100% à la frontière exacte, décroît à 0% à 2×rayon. Les grains mélangent alors des pixels quasi-identiques → jointure imperceptible.
Rayons disponibles	OFF (désactivé) / 48 px (moyen) / 64 px (large) / 96 px (très large) / 128 px (max). Le rayon est la demi-vie : à cette distance, 85% des pixels sont de la source dominante.
Portée effective	La transition couvre environ 3×rayon de chaque côté de la frontière. Avec 96 px : ~580 px de transition soit 1,5 cm à ZL17 — imperceptible en vol.

Comment utiliser la Fusion Preview

Élément de l'interface	Description
Grand visualiseur (560×560 px)	Affiche en haut la prévisualisation du rayon sélectionné sur le DDS courant
Vignettes horizontales	5 vignettes en rangée sous le visualiseur : OFF / 48 px / 64 px / 96 px / 128 px
Affichage RGB A/B	Moyennes colorimétriques mesurées de chaque source (hors zone de transition)
Superposition rouge/bleu	Rouge = pixels qui seront source A dans la zone de transition, bleu = source B — visualise la densité de dispersion
Clic simple sur vignette	Agrandit la vignette dans le visualiseur pour voir le détail
Double-clic sur vignette	Sélectionne ce rayon et ferme la fenêtre — appliqué au Build suivant

⚠ Ne pas utiliser Color Check et Color Normalize simultanément avec un référent temporaire — les corrections s'additionnent.

12.4 Workflow Color Check complet

11. Cliquer Color Check... → la fenêtre s'ouvre, Enable se grise, Color Normalize se désactive
12. Section ① : attendre la fin du scan — X dominantes sur Y DDS
13. Corriger les dominantes si nécessaire (curseurs R/G/B ou Auto-détecter)
14. Section ② : cliquer sur un DDS à jointure visible → Fusion Preview s'ouvre
15. Sélectionner le rayon optimal par double-clic (64 ou 96 px recommandé pour les frontières)
16. Supprimer les DDS concernés → Lancer Build
17. Rescanner pour vérifier → zéro dominante → fermer → Color Normalize se réactive

12.5 Archive .ccorr

- Les corrections sont sauvegardées dans color_corrections.ccorr dans textures/ de la tuile.
- Archive automatique dans Color_check/ à chaque Build.
- Restauration possible depuis Color_check/ en cas de reconstruction complète de la tuile.

10. Workflow recommandé et validé

Pour une tuile frontière (ex : +50+003 Belgique/France)

18. Configurer Ortho4XP.cfg avec les paramètres validés :

```
mask_width=100
imprint_masks_to_dds=False
ratio_water=0.2
water_tech=XP12
```

19. Activer Maritime Mask (50 px recommandé) avant Draw Water Masks.
20. Section ② Color Check : sélectionner le rayon optimal (64 ou 96 px pour les frontières).
21. Supprimer les DDS existants de la tuile avant reconstruction.
22. Lancer Build Imagery/DSF — les corrections s'appliquent automatiquement.
23. Valider dans X-Plane : disparition des rectangles en mer + harmonisation des couleurs à la frontière.

Commandes utiles (macOS/Linux)

```
# Vérifier les masques existants
find ~/Applications/ORTHO4XP_V3/Masks -name '*46*003*' 2>/dev/null

# Supprimer les DDS d'une tuile pour reconstruction
rm ~/Applications/ORTHO4XP_V3/Tiles/+50+003/textures/*.dds
```

— PARTIE 2 — TECHNIQUE

Architecture, pipeline, algorithmes — pour développeurs et contributeurs

8. Processus interne JPG → PNG → DDS

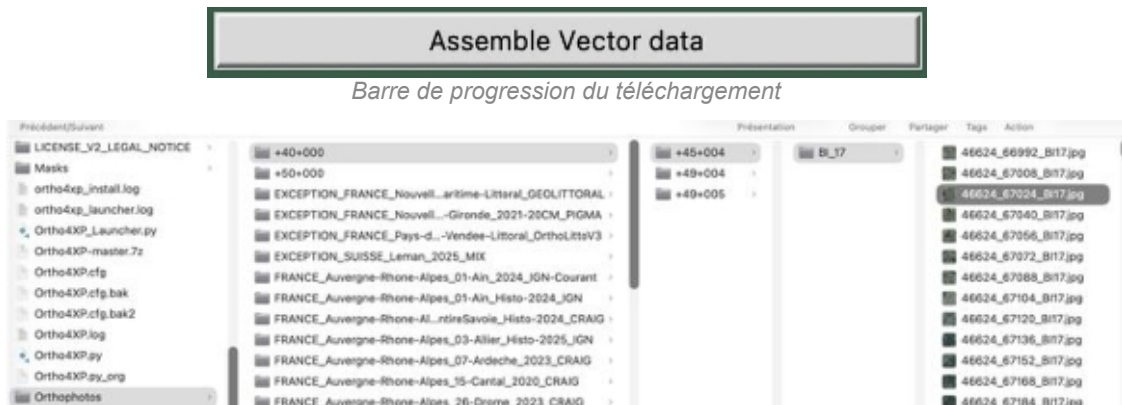
Cette section décrit le processus interne d'Ortho4XP pour transformer les orthophotos sources en textures DDS utilisées par X-Plane.

4.1 Cas 1 — Un seul fournisseur d'image

Scénario le plus simple : toutes les orthophotos de la tuile proviennent d'un même fournisseur.

Étape 1 — Téléchargement des JPG sources

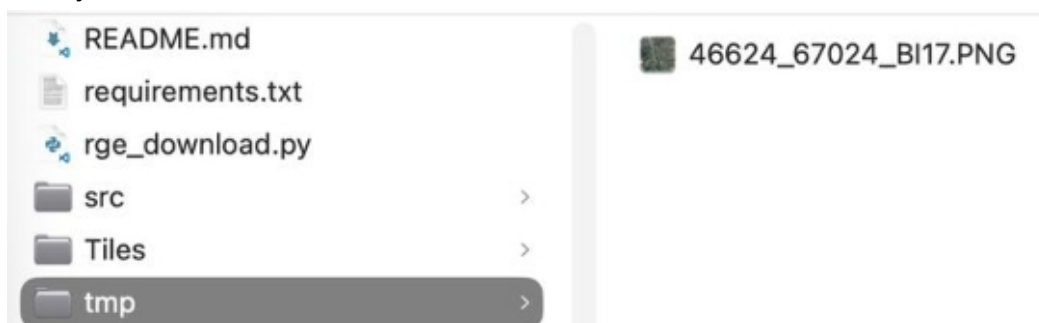
Ortho4XP télécharge les JPG et les place dans le dossier Orthophotos/, dans un sous-dossier par fournisseur et tuile.



Structure du dossier Orthophotos/ — JPG sources classés par fournisseur et tuile

Étape 2 — Conversion JPG → PNG en mémoire (dossier tmp/)

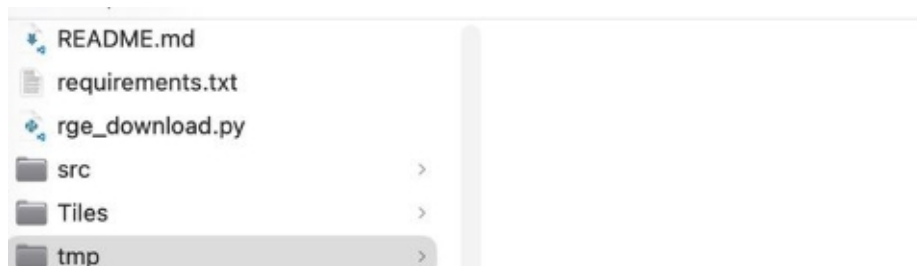
Pendant le Build, Ortho4XP lit chaque JPG source, applique Color Normalize en mémoire (correction colorimétrique individuelle), puis enregistre le résultat temporairement en PNG dans le dossier tmp/. Le JPG source n'est jamais modifié.



PNG temporaire créé dans tmp/ pendant le Build

Étape 3 — Conversion PNG → DDS

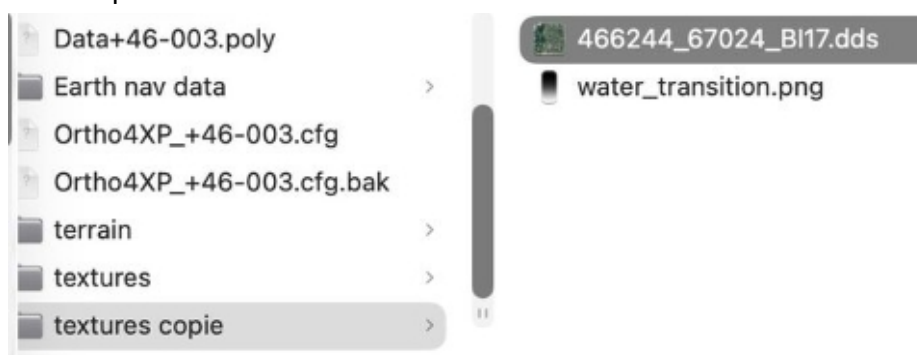
Ortho4XP appelle nvcompress pour convertir le PNG en DDS (format DXT1 sans flag sRGB). Le PNG temporaire est aussitôt supprimé.



Conversion PNG → DDS via nvcompress

Étape 4 — Placement du DDS dans textures/

Le DDS final est placé dans Tiles/+XX+XXX/textures/. X-Plane le lit directement. La procédure recommence pour l'orthophoto suivante.



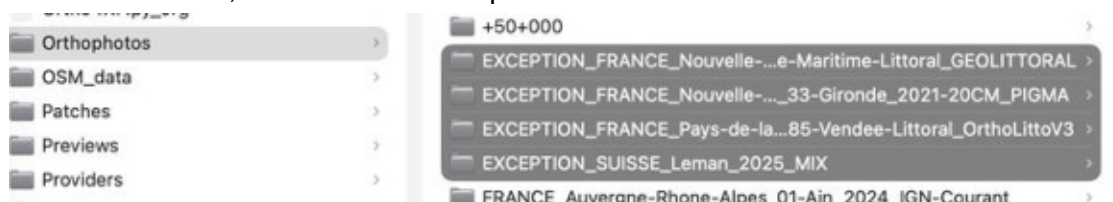
DDS placé dans le dossier textures/ de la tuile

4.2 Cas 2 — Plusieurs fournisseurs d'image (cas frontière)

Cas plus complexe : les orthophotos proviennent de sources différentes (ex : France IGN + Belgique NGI). C'est le cas typique des tuiles frontières comme +50+003.

Étape 1 — JPG dans des sous-dossiers distincts par fournisseur

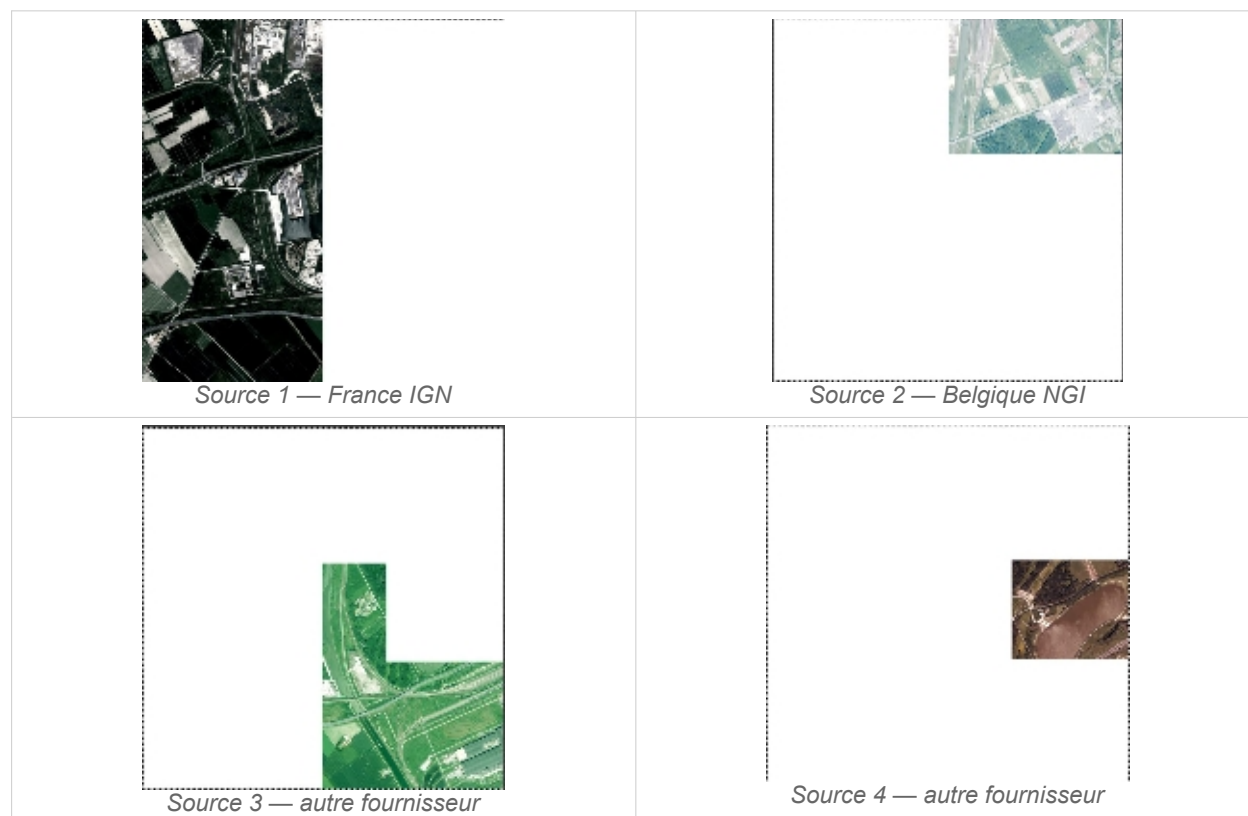
Chaque fournisseur a son propre sous-dossier dans Orthophotos/. Les JPG ont des colorimétries différentes selon la saison, la calibration du capteur et le traitement du fournisseur.



Dossier Orthophotos/ avec plusieurs sous-dossiers fournisseurs distincts

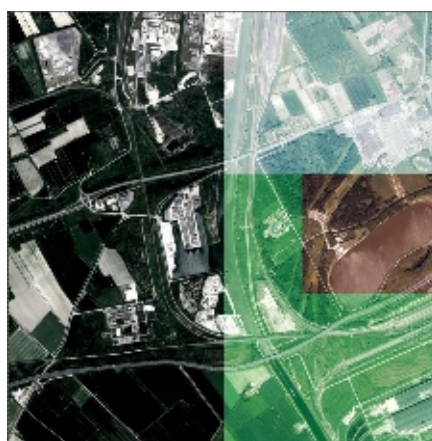
Étape 2 — Sources avec colorimétries différentes

Exemple réel avec 4 sources différentes. Sans correction, les différences de teinte, luminosité et gamma sont immédiatement visibles — les jointures seront brutales dans X-Plane.



Étape 3 — Assemblage PNG : jointures brutales sans correction

Sans Color Normalize, Ortho4XP assemble les sources telles quelles en un seul PNG. Les différences de colorimétrie créent des jointures parfaitement visibles, correspondant exactement aux limites administratives ou zones d'acquisition.



Assemblage brut de 4 sources — jointures brutales visibles sans correction colorimétrique

⚠ C'est exactement ce problème que corrige Color Normalize V2 : chaque source est normalisée individuellement vers le cube de référence calibré, puis les deux sources sont fondues avec une dispersion exponentielle (grains de sable) pour masquer toute limite visible.

Étapes 4 et 5 — identiques au cas 1

Une fois le PNG assemblé et corrigé, la conversion en DDS et le placement dans textures/ suivent le même processus que le cas 1.

9. Fichiers spéciaux — Pour le développeur

6.1 Pourquoi ces fichiers existent

Ortho4XP V3.0 est distribué avec un système d'installation en 2 étapes. L'objectif : l'utilisateur ne touche jamais au terminal.

Ces fichiers permettent deux choses : (1) créer des lanceurs natifs exécutables sans Python préinstallé, (2) installer automatiquement tout l'environnement Python isolé dans venv/.

Fichier	Rôle	Exécuté par
create_launcher_prerequis.py	Génère Lanceur_Installation_Prerequis.app / .vbs / .desktop selon la plateforme	Adaptation et évolution et modernisation Roland — 1 seule fois avant distribution
Lanceur_Installation_Prerequis.app	Lance INSTALL_PREREQUIS.py — binaire C universel, pas besoin de Python	Utilisateur — premier double-clic
INSTALL_PREREQUIS.py	Fenêtre verte — détecte le système, installe Python/Homebrew/venv/modules	Lanceur_Installation_Prerequis
create_launcher_ORTHO.py	Génère Lanceur ORTHO4XP.app / .vbs / .desktop via le bouton du Launcher	Utilisateur via bouton Launcher
Lanceur ORTHO4XP.app	Lance directement Ortho4XP_Launcher.py via venv — usage quotidien	Utilisateur — double-clic quotidien
Ortho4XP_Launcher.py	Interface principale — installation dépendances et lancement Ortho	Lanceur ORTHO4XP

10. Fichiers Ortho4XP modifiés et ajoutés

10.1 Nouveaux fichiers V3

Fichier	Version	Rôle
create_launcher_prerequis.py	V3.0 — Mars 2026	Génère le lanceur installation (binaire C) multiplateforme
create_launcher_ORTHO.py	V3.0 — Mars 2026	Génère le lanceur quotidien via bouton Launcher
INSTALL_PREREQUIS.py	V3.0 — Mars 2026	Bootstrap installation : Python, Homebrew, venv, modules
O4_Saturation_Window.py	V3.0 — Mars 2026	Fenêtre Saturation avec aperçu Avant/Après en temps réel
O4_Color_Apply.py	V3.0 — 30 Mars 2026	Applique les corrections .ccorr APRÈS Color Normalize, AVANT DDS
O4_Color_Normalize.py	V3.0 — 31 Mars 2026	Normalisation calibrée sur 48 753 JPG Europe
O4_Color_Check.py	V3.0 — 31 Mars 2026	Interface vérification et correction jointures DDS
O4_Version.py	V3.0 — Mars 2026	Chaîne de version : V3.0 - sRGB Roland Edition
O4_Sea_Texture.py	V3.2 — Mai 2026	Correction textures mer côtières — fond marin via JPG-Patch (smearing 15+15 px, undulation, pipeline V50+)
O4_Coastal_Manager.py	V3.2 — Avril 2026	Post-traitement masques côtiers : bord vague/écume (ECUME_WIDTH=30 px, NOISE=20 px) — module autonome
O4_Benchmark.py	V3.2 — Mai 2026	Benchmark CPU/GPU, Timeline, DebugViz — module autonome (zéro cassure V2)

8.2 Fichiers modifiés — correctifs Python 3.12

Fichier	Correction
O4_DSF_Utills.py	Fix UnboundLocalError bDEMN/bDEMS — initialisation b" avant boucle while
O4_DEM_Utills.py	rasterio remplace osgeo.gdal — autonome dans venv, tuile +45+004 sans falaises 
O4_Bathymetry.py	import os ajouté (manquant Python 3.12)
O4_Recut_Water.py	import numpy ajouté (manquant Python 3.12)
O4_Scenery_Utills.py	import os ajouté (manquant Python 3.12)
O4_UI_Utills.py	import os ajouté (manquant Python 3.12)
O4_Overlay_Utills.py	import time ajouté (manquant Python 3.12)
O4_Geotag.py	import os ajouté + Transformer.from_crs().transform() corrigé (API pyproj 3.x)
Providers/Global/OSM.lay	URL → tile.openstreetmap.de + User-Agent conforme politique OSM

8.3 Fichiers modifiés — fonctionnalités V2

Fichier	Modifications principales
O4_GUI_Utils.py	Color Normalize 2 lignes. Maritime Mask + curseur 30-500 px. Carte bouton droit. OACI thread-safe. Config singleton.
O4_GUI_Utils.py	Overlay aéroports OACI — thread Overpass + polling canvas.after() + tag_raise(). Scroll clic gauche ajouté (B1-Motion).
O4_Imagery_Utils.py	User-Agent Chrome 122. Fondu grains de sable exponentiel. Correction croisée vers cible commune.
O4_Config_Utils.py	Correction 3 bugs sauvegarde : zone_list, update_tile_cfg manquant, chemins incohérents → Tile.write_to_config().
O4_Mask_Utils.py	Option Maritime Mask. Curseur réglable 30–500 px.
O4_DSF_Utils.py	DECAL_LIB tri_type==0 uniquement → rectangles bleus éliminés.
Ortho4XP_Launcher.py	Interface 2 colonnes, menu installation Mac/Linux/Win, rasterio, boutons ttk uniformes.
O4_GUI_Utils.py	Fournisseur personnel V3.2 : fenêtre ajout/modification/suppression providers TMS personnels (dossier Personnel/). Validation nom + URL. Sélection immédiate dans la liste Imagery après sauvegarde.
O4_Color_Normalize.py	v2.5 (V3.2) : appel Coastal Manager post-normalisation. Rayon adaptatif local ($\Delta E > 25$ pts, max $\times 2.0$). Anti-HDR seams (crushed-blacks/halos). ZL17–20 facteurs affinés. Support ZonePhoto.comb. Correction adaptative ZL (force 0.30→0.18→0.08 selon ZL).

11. Modules Python et versions venv/

Version Python : 3.12.x | Architecture : arm64 (Apple Silicon) ou x86_64 (Intel / Linux / Windows)

Module	Version validée Mars 2026	Rôle dans Ortho4XP V2
numpy	2.4.3	Calculs matriciels mesh, altimétrie, colorimétrie
Pillow	12.1.1	Lecture/écriture JPEG/DDS, Color Normalize, Color Check
psutil	7.2.2	Sentinelle CPU/RAM au lancement
pyproj	3.7.2	Conversion systèmes de coordonnées géographiques
fiona	1.10.1	Lecture fichiers géospatiaux (inclut osgeo si présent)
scipy	1.17.1	Calculs scientifiques pour mesh + distance_transform_edt
shapely	2.1.2	Géométrie vectorielle — découpage côtes et routes
rtree	1.4.1	Index spatial R-tree pour les requêtes géographiques
scikit-fmm	2025.6.23	Génération masques d'eau (Water masks)
customtkinter	5.2.2	Widgets UI modernes (boutons, sliders)
rasterio	dernière stable	Lecture fichiers TIF altimétrie — remplace osgeo.gdal système
requests	dernière stable	Téléchargement tuiles imagery (HTTP)
certifi	dernière stable	Certificats SSL pour les connexions HTTPS
urllib3	dernière stable	Client HTTP bas niveau

✔ *rasterio inclut ses propres librairies GDAL compilées. Aucune installation GDAL système requise sur aucune plateforme.*

|

12. Architecture des fichiers ORTHO4XP_V3/

Structure du dossier après installation complète :

```
ORTHO4XP_V3/
├─ create_launcher_prerequis.py    ← génère Lanceur_Installation_Prerequis.app
├─ create_launcher_ORTHO.py        ← génère Lanceur ORTHO4XP.app
├─ INSTALL_PREREQUIS.py            ← bootstrap installation (fenêtre verte)
├─ Ortho4XP_Launcher.py            ← launcher principal
├─ Ortho4XP.py                     ← application principale
├─ Lanceur_Installation_Prerequis.app
├─ Lanceur_ORTHO4XP.app
├─ Ortho4XP.cfg                    ← configuration globale
├─ venv/                           ← environnement Python isolé
├─ Color_check/                    ← archives .ccorr
├─ Tiles/+XX+XXX/textures/         ← DDS générés
├─ Orthophotos/+XX+XXX/PROV_ZL/    ← JPG sources
├─ Masks/                          ← masques eau
├─ Providers/                      ← fournisseurs (.lay, .comb)
├─ Extents/                        ← zones géographiques (.ext)
├─ Utils/                          ← binaires natifs (nvcompress, etc.)
├─ src/                            ← fichiers Python sources
│   ├── O4_GUI_Utils.py
│   ├── O4_Config_Utils.py
│   ├── O4_Imagery_Utils.py
│   ├── O4_Color_Normalize.py
│   ├── O4_Color_Check.py
│   ├── O4_Color_Apply.py
│   ├── O4_Mask_Utils.py
│   └─ O4_DSF_Utils.py
```

13. Historique des versions et modifications

Version	Modifications et ajouts
v1–v2	Portage Python 3, interface Tkinter de base, téléchargement orthophotos. Création O4_Color_Normalize.py et O4_Color_Check.py. Workflow : suppression DDS + mémorisation + rebuild.
v3–v7	Scan parallèle 8 threads, filtrage DDS dominantes, désactivation auto Color Normalize ↔ Color Check.
v8–v12	Bouton Exporter, référent global/temporaire, Bouton Lancer Build, adaptation 4K, premier cube de normalisation colorimétrique.
v13	Session Mars 2026 : corrections bugs multiples, GDAL automatique, Build DSF validé X-Plane 12 
v14	GDAL macOS Homebrew 3.9.2, correction DEM View, split('=',1), curseur Netteté, champs width=40, minsize(1320×860), bouton GDAL dédié Launcher.
v15 — 24 Mars 2026	Installation autonome complète (binaire C universel arm64+x86_64), rasterio 100% autonome. Python 3.12 : 7 fichiers corrigés. 2 lanceurs natifs. Color Normalize 2 lignes. Preview OSM (tile.openstreetmap.de). Saturation Window Avant/Après. Config singleton. Overlay OACI → Overpass (thread, 3 serveurs). Scroll clic gauche.
v16 — 30 Mars 2026	O4_Color_Apply.py : corrections .ccorr APRÈS Color Normalize. Color Normalize par source individuelle dans combine_textures(). Color Check v2.3 : section ① + section ② (grains de sable, preview). Archive .ccorr → Color_check/. DSF : DECAL_LIB tri_type==0 → rectangles bleus éliminés. Cube recalibré 48 753 JPG : R=86.5 G=96.5 B=86.9, gamma=0.63, CORRECTION_STRENGTH=0.30.
v17 — 2 Avril 2026	Correction 3 bugs sauvegarde cfg (O4_Config_Utills.py). Fondu grains de sable renforcé : correction croisée vers cible commune 50/50. Color Check FusionPreview : vignettes rangée horizontale. Maritime Mask curseur 30–500 px validé.
v18 — Juin 2026	Passage V3.2 (Mai–Juin 2026). O4_Sea_Texture.py : correction textures mer côtières via JPG-Patch (smearing 15+15 px, undulation AMP=12 px FREQ=60 px). O4_Coastal_Manager.py v2.0 : post-traitement masques côtiers (ECUME_WIDTH=30 px, NOISE=20 px). Fournisseur personnel : fenêtre GUI ajout/modification/suppression providers TMS (dossier Personnel/). O4_Benchmark.py : module benchmark CPU/GPU + Timeline + DebugViz. Correction 2 bugs historiques V1 : (1) masque côtier empiétant en mer → missing JPG → DDS bloqué corrigé ; (2) .ter généré sans DDS en pleine mer corrigé (suppression sea_overlay). Overlay OACI : création PNG liste aéroports par tuile (thread Overpass). O4_Color_Normalize v2.5 : appel Coastal Manager post-normalisation, rayon adaptatif local ($\Delta E > 25$ pts), anti-HDR seams, support ZonePhoto.comb.

— PARTIE 3 —

LICENCE & CRÉDITS

Conditions d'utilisation, responsabilités et remerciements

14. Avertissement légal — ORTHO4XP_V3

La version FRANÇAISE ci-dessous est la seule version officielle faisant foi.

I. VERSION FRANÇAISE

DESCRIPTION ET CRÉDITS

Ortho4XP_V2 est une évolution de la version ORTHO4XP_Master 1.40. Concept & design : Roland (Ypsos) | Codage : Claude (Anthropic AI). Basé sur les travaux originaux d'Oscar Pilote (<https://github.com/oscarpilote/Ortho4XP>).

ACCEPTATION DES RISQUES ET RESPONSABILITÉ

L'installation, la configuration et l'utilisation du logiciel ORTHO4XP_V3 s'effectuent sous la SEULE ET ENTIÈRE RESPONSABILITÉ DE L'UTILISATEUR.

PROTECTION COLLECTIVE DES INTERVENANTS

- Aucun membre de la communauté ne pourra être tenu pour responsable d'un quelconque dommage direct ou indirect.
- Aucun dédommagement ne pourra être exigé auprès des contributeurs en cas de préjudice subi.
- L'utilisateur renonce expressément à toute poursuite judiciaire à l'encontre de la communauté.

ABSENCE DE GARANTIE (GPL v3, Articles 15 & 16)

Le programme est fourni « TEL QUEL », SANS AUCUNE GARANTIE. L'intégralité du risque incombe à l'utilisateur.

LOGICIELS TIERS

Ce logiciel utilise des composants tiers (Medit, Triangle, DSFTool, 7-Zip, NVIDIA Texture Tools) soumis à leurs propres notices légales.

Note : Effectuez toujours une sauvegarde de votre système avant utilisation.

II. ENGLISH VERSION (TRANSLATION — NON-BINDING)

DESCRIPTION AND CREDITS

Ortho4XP_V2 is an evolution of ORTHO4XP_Master 1.40. Concept & Design: Roland (Ypsos) | Coding: Claude (Anthropic AI). Based on original works by Oscar Pilote.

USER RESPONSIBILITY & NO WARRANTY (GPL v3)

The installation and use of ORTHO4XP_V3 are at the user's sole and entire responsibility. The program is provided "AS IS", WITHOUT WARRANTY OF ANY KIND.

15. Remerciements

Document généré Roland (Ypsos) — Mars 2026



Remerciements pour tests

Equipe Développeurs Ortho XPC

Cricri alias Odevillée

Jojo : conseils en procédure de fabrication tuiles Orthophoto, et contributeur a OscarPilote pour la version 1

Henri, Test Mac